Introduction to C++

Benn Tan

NUS Hackers

Hackerschool

7 Apr 2026

About Me

- Year 3 studying Computer Science
- Been writing C++ for fun for a while
- Enjoys (physical) running outside of running code

C++ Fundamentals

- Systems language → fast, low-level control
- Manual memory mamagement
- Used in browsers, databases, game engines

👉 Goal: build intuition, not memorize syntax

Getting started

- On Mac, you can run `xcode-select --install`
- On Windows, you can install Dev-C++ IDE.
- On Linux, you can install via `sudo apt update g++`
- To avoid wasting time wrestling with installation issues, you can also use an online compiler like *JDoodle*

Program Execution Model

Source code → compiled → executable

The source code is contained in file called `main.cpp` like

```
#include <iostream>

int main() {
    std::cout << "Hello, world!";
    return 0;
}
```

Compiling

To run it, we can compile the code like so:

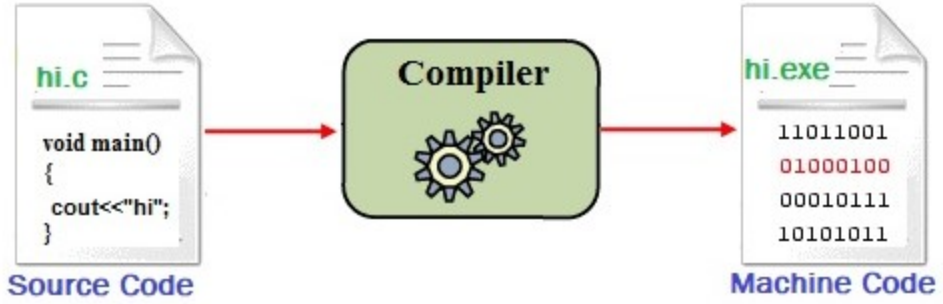
```
clang++ main.cpp -o hello-world
./hello-world
```

Note

Each time you make changes to the source code, you will need to re-compile the code before executing it!

If you're curious

Think of a compiler as a black box that simply **converts** source code to machine code.



Basic Syntax & I/O

Anatomy of a C++ Program

Recall the source code you previously saw:

```
#include <iostream>

int main() {
    std::cout << "Hello, world!";
    return 0;
}
```

Breaking Each Part Down

Breaking it down

Code	What it does
`#include <iostream>`	Enables input/output (`std::cout`)
`int main() {`	Program entry point
`std::cout << "Hello World";`	Prints to console
`return 0;`	Ends program (0 = success)

Mini-Exercise

Try augmenting the 'Hello, world!' string to something else like *your name*.

(Lack of) Newlines

Realize that if you try printing multiple times, you will notice that the output gets concatenated on the same line.

```
#include <iostream>

int main()
{
    std::cout << "benn";
    std::cout << "tan";
    std::cout << "jia";
    return 0;
}
```

Code Output

```
2:41:04 [~/Desktop/cpp]
❯ clang++ main.cpp -o hello-world

2:42:29 [~/Desktop/cpp]
❯ ./hello-world
benntanjia%
```

Adding newlines

To add newlines, we can suffix each `std::cout` with a `std::endl` (or a `\n`)

```
#include <iostream>

int main()
{
    std::cout << "benn" << std::endl;
    std::cout << "tan" << std::endl;
    std::cout << "jia";
    return 0;
}
```

Code Output

```
2:42:31 [~/Desktop/cpp]
❯ clang++ main.cpp -o hello-world

2:43:31 [~/Desktop/cpp]
❯ ./hello-world
benn
tan
jia%
```

Input

Like with any programs, we can augment it to accept input so that we don't have to hardcode the values.

To read input, we can use `std::cin >>`

Code Example

You can remember the syntax as **data flowing into x**

```
#include <iostream>

int main() {
    int x;
    std::cin >> x;
    std::cout << "You entered: " << x << std::endl;
}
```

Namespaces

If you find prefixing `std::` before every function invocation cumbersome, you can use `using namespace std` at the top of the program.

```
#include <iostream>
using namespace std;
int main() {
    int x;
    cin >> x;
    cout << "You entered: " << x << endl;
}
```

Mini-Exercise

Build a program that takes in and prints the result of the summation of two numbers

Variables

What ?

In the previous code, you already had a glimpse into variables.

In essence, a variable is simply an named container that stores some value.

Breaking It Down

```
int x = 5;
```

Part	Details
`int`	Indicates the **type** of the variable (integer)
`x`	The **name** of the variable
`5`	The **value** assigned to the variable

Common Data Types

Type	Code	Description
`int`	`int age = 24;`	Integer (whole numbers)
`double`	`double pi = 3.14;`	Floating point numbers
`char`	`char x = 'c';`	Singular character
`bool`	`bool isGood = true;`	True / False
`std::string`	`std::string name = "Benn";`	Sequence of characters

Declaration vs Initialisation

Declaration is when we define a variable's type and name without assigning it a value yet.

Initialisation is when we give it a value.

```
int x;       // declaration
x = 10;      // assignment

int y = 10; // declaration + initialisation
```

Arithmetic Operators

You can also perform arithmetic operations just like how you would do in normal math:

```
int x = 10;
int y = 12;

int sum = x + y;
int difference = x - y;
int product = x * y;
double quotient = x / y; // note that we use the double here
int remainder = x % y;

int expression = ((x + y) * (x - y));
```

⚠️ Common Pitfall

Integer division truncates the division

So `10 / 12 = 0` instead of `0.83`

Control Flow

So far, our programs have executed line-by-line, step-by-step from top to bottom. In the real world however, we might want to

- make decisons based on a condition
- repeat certain actions
- organise logic into reusable pieces

Conditionals

Conditionals let our program make decisions.

```
int score = 85;

if (score == 100) {
    std::cout << "Perfect" << std::endl;
} else if (score >= 90) {
    std::cout << "Grade A" << std::endl;
} else if (score >= 80) {
    std::cout << "Grade B" << std::endl;
} else if (score >= 70) {
    std::cout << "Grade C" << std::endl;
} else {
    std::cout << "Needs improvement" << std::endl;
}
```

⚠️ Common Pitfall

`=` means assignment while `==` checks for equality

```
int x = 5;        // assignment
x == 5;           // comparison
```

Loops

While loop

A while loop keeps running as long as the condition is fufilled.

```
int x = 1;

while (x <= 5) {
    std::cout << x << std::endl;
    x++;
}
```

For loop

A for loop is useful when we know how many times we want to repeat something.

```
for (int i = 1; i <= 5; i++) {
    std::cout << i << std::endl;
}
```

`break` keyword

`break` forces the loop to terminate prematurely.

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        break;
    }
    std::cout << i << std::endl;
}
```

`continue` keyword

`continue` skips the rest of the current iteration.

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        continue;
    }
    std::cout << i << std::endl;
}
```

Functions

Basic Usage

Functions let us group code into reusable blocks.

```
#include <iostream>

void greet() {
    std::cout << "Hello!" << std::endl;
}

int main() {
    greet();
}
```

Breaking it down

- `void` means the function does not return anything
- `greet` is the function name
- `greet()` calls the function, causing Hello! to be printed.

Function Parameters

Our current `greet` function always print the same message (which isn't very flexible). Let's augment it by adding a parameter so that our greeting message can change depending on the person.

```
#include <iostream>

void greet(std::string name) {
    std::cout << "Hello, " << name << "!" << std::endl;
}

int main() {
    greet("Benn");
    greet("Bryan");
    greet("Anton Tim");
}
```

Functions Return Value

A return value is the value the function sends back to the caller after it's done executing. At the call site, you can use the result however you wish.

```
#include <iostream>

int add(int x, int y) {
    return x + y;
}

int main() {
    int result = add(3, 4);
    std::cout << result << std::endl;
}
```

Exercise(s)

⭐ Write a function countdown(int n) that takes in a number n and prints from n down to 1.

⭐⭐ Write a program that hardcodes a secret number. It will repeatedly ask the user to guess. Depending on the user's guess, the program will feedback Too low, Too high or Correct.

Memory Ownership

Memory Ownership

So far, we've been using variables without really thinking **where they live / who owns them**.

In C++, understanding memory is extremely important as it helps you write

- faster programs
- safer code
- avoid nasty bugs like memory leaks

Pointers

Address of Variables

So far, we've been working with variables like:

```
int x = 10;
```

But sometimes, we don't want the value itself, rather we will want to know **where is this value stored in memory**?

The variable x could have some address like `0x100...`

Visually...

We can use the address-of operator (the `&`) to find out!

Code Snippet

```cpp
int main()
{
    int x = 10;
    std::cout << "location of x: " << &x << std::endl;
}
```

```
23:45:55 [~/Desktop/cpp]
❯ clang++ -std=c++17 -Wall -Wextra -pedantic-errors main.cpp -o hello-world

23:46:05 [~/Desktop/cpp]
❯ ./hello-world
location of x: 0x16b1058ac
```

Type of Pointers

We know the type of `x` is an `integer`, but what's the type of `&x`?

Instead of second guessing ourselves, let's rely on the C++ compiler to tell us. C++ offers a `typeid` function that seems useful:

```
int main()
{
    int x = 10;
    std::cout << typeid(x).name() << std::endl;
    std::cout << typeid(&x).name() << std::endl;
}
```

Code Output

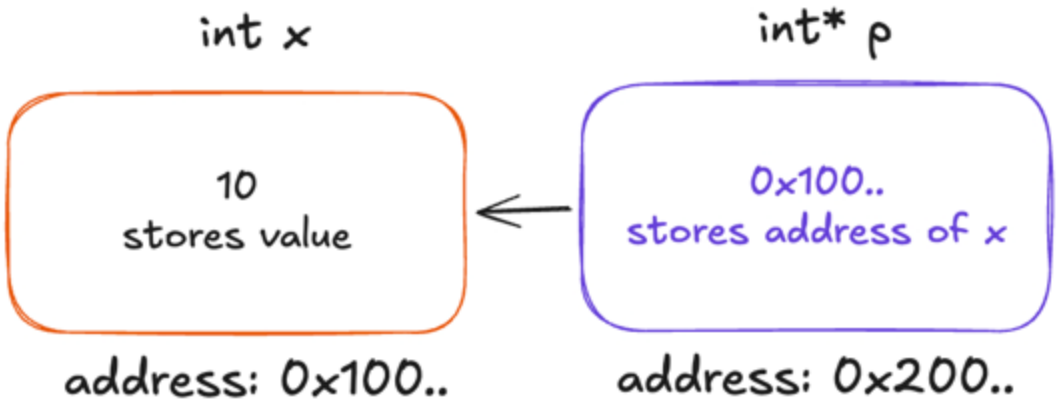
```
23:51:15 [~/Desktop/cpp]
❯ clang++ -std=c++17 -Wall -Wextra -pedantic-errors main.cpp -o hello-world

23:51:21 [~/Desktop/cpp]
❯ ./hello-world
i
Pi
```

The output may look cryptic but:

- `i` (as you might have guessed) represents int
- `Pi` (which you also might have guessed) represents pointer to int (int*)

Visual Aid

```
int main()
{
    int x = 10;
    int* p = &x;
}
```



Motivation behind de-referencing

Given a pointer of type (`int *`), is there a way I can get the actual value (and not the address) it points to?

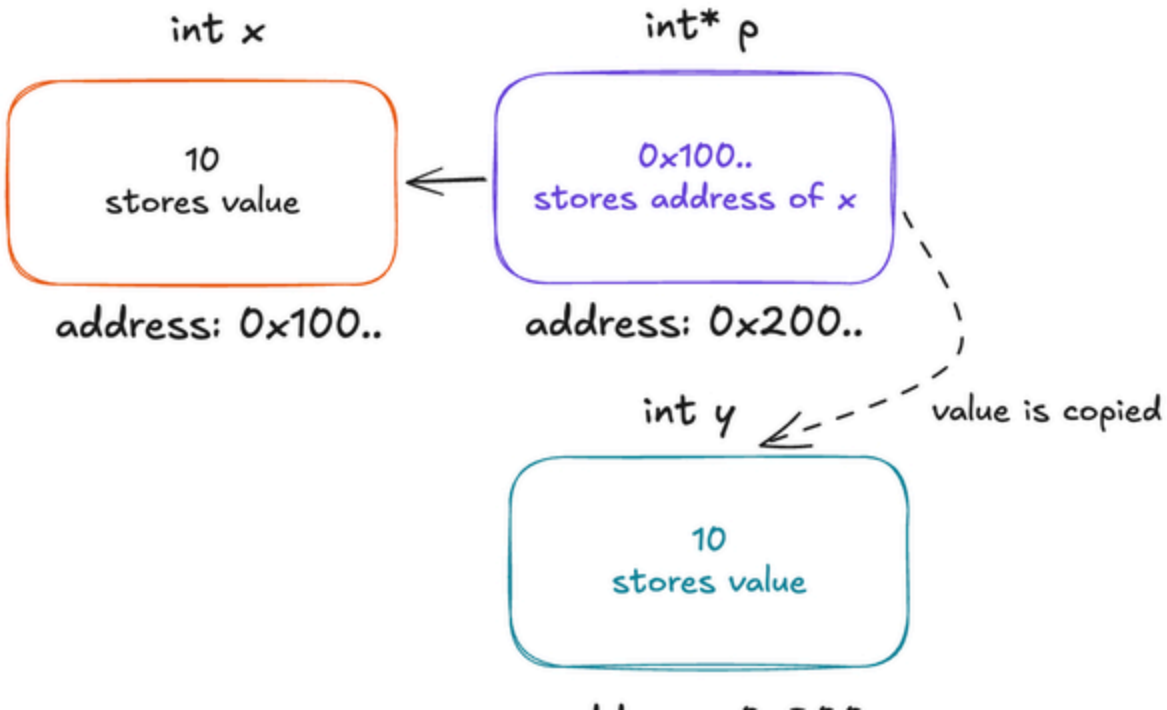
```
int main()
{
    int x = 10;
    int* p = &x;

    // can i get 10 using variable p alone?
}
```

De-referencing

Yes we can! Since the pointer `p` stores an address, we can dereference the pointer using `*` to get the value! Think of dereferencing as following the address to get the value stored there.

Code Snippet

```
int main() {
    int x = 10;
    int* p = &x;
    int y = *p;
}
```



Observation

Realise while both values of x and y are the same, they are not pointing to the same `10`. They each contain their own version of `10` (and possess their own memory addresses).

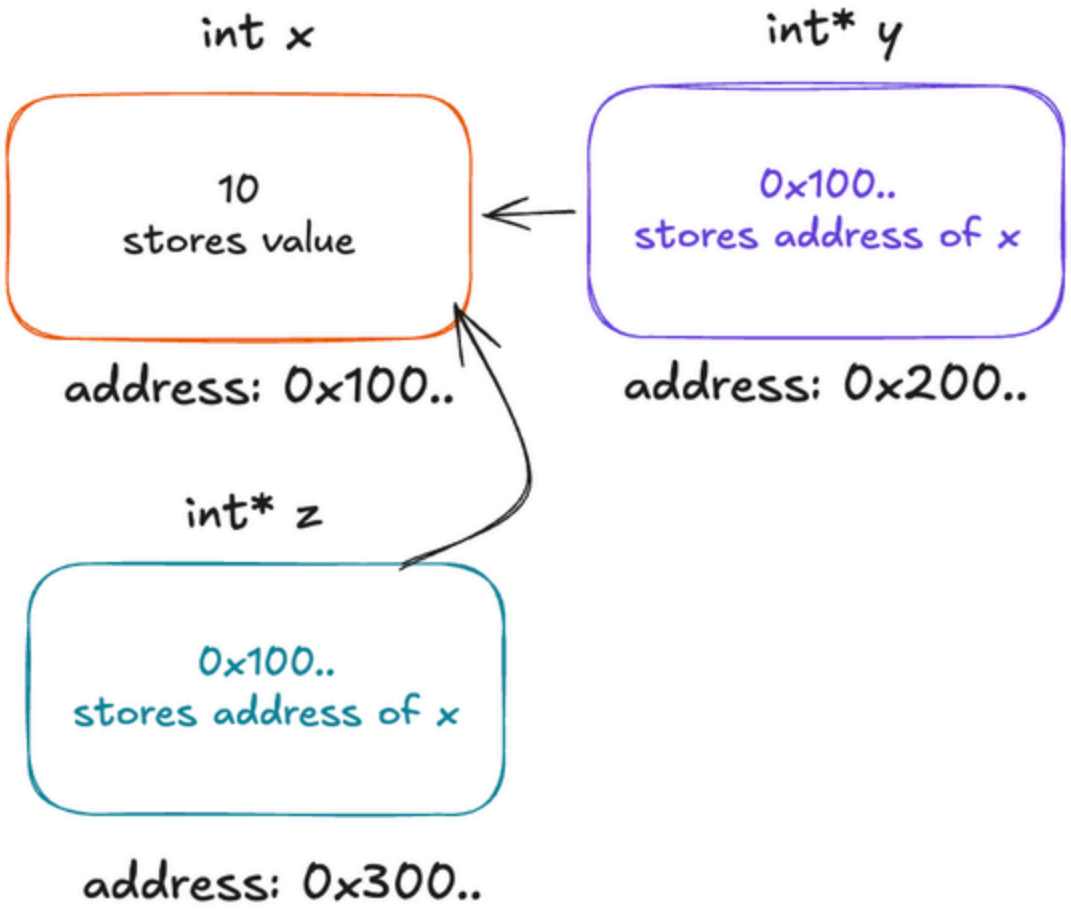
Note that this behaviour diverges with arrays / strings as we will see.

Qn: How do we make `x` and `y` point to the same 10?

Qn: How do we make `x` and `y` point to the same 10?

```
#include <iostream>

int main() {
    int x = 10;
    int* y = &x;   // y points to x
    int* z = &x;   // z also points to x

    std::cout << x << std::endl;   // 10
    std::cout << *y << std::endl;  // 10

    *y = 20;  // modify via pointer

    std::cout << x << std::endl;   // 20 (x changed!)
}
```

Visual Aid


int x
10
stores value
address: 0x100..
int* y
0x100..
stores address of x
address: 0x200..
int* z
0x100..
stores address of x
address: 0x300..

Pointer-Ception

```cpp
#include <iostream>

int main() {
    int x = 10;             // normal variable
    int* p = &x;            // pointer to x
    int** pp = &p;          // pointer to pointer

    std::cout << "x value: " << x << std::endl;
    std::cout << "Address of x (&x): " << &x << std::endl;

    std::cout << "p (points to x): " << p << std::endl;
    std::cout << "Value at p (*p): " << *p << std::endl;

    std::cout << "pp (points to p): " << pp << std::endl;
    std::cout << "Value at pp (*pp): " << *pp << std::endl;
    std::cout << "Value at *pp (**pp): " << **pp << std::endl;

    return 0;
}
```

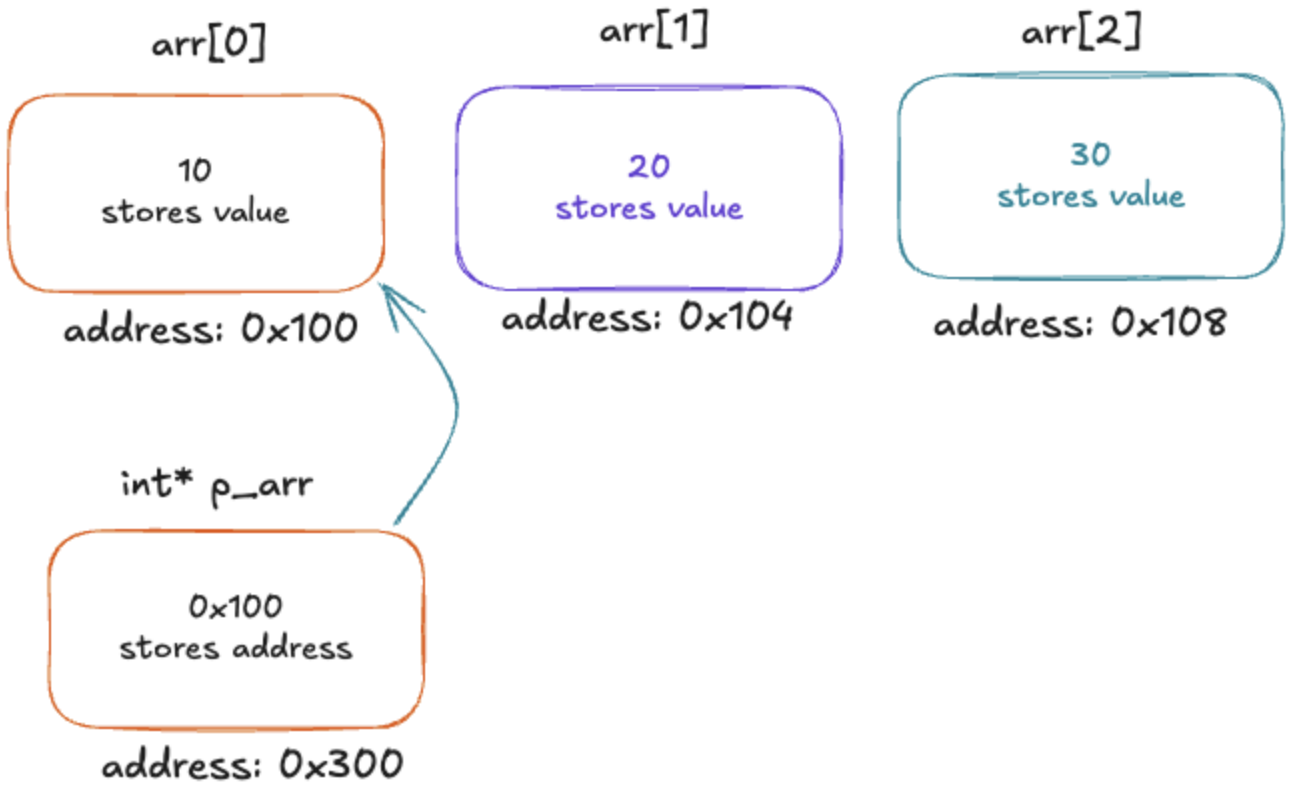
Arrays & Strings

Arrays

To declare an array:

```
int arr[3] = {10, 20, 30}; // array
int* p_arr = arr;          // pointer to array
```

Visual Aid


arr[0]
10
stores value
address: 0x100
arr[1]
20
stores value
address: 0x104
arr[2]
30
stores value
address: 0x108
int* p_arr
0x100
stores address
address: 0x300

Observations

Note that:

- All 3 elements sit **side-by-side** (i.e there exists no gaps between them). This is what we mean by **contiguous memory**.
- You may also have noticed that the difference between each memory address is 4 bytes. This because (usually) `int` occupies 4 bytes.

Pointer Arithmetic

Since `p_arr` stores the address of the first element, you can access `arr[0]` via dereferencing `*p_arr`.

What about `arr[1]`? The cool thing about pointer arithmetic is that it's type-aware, you can simply perform `(p_arr + 1)` to move forward by 1 element of type `int` (i.e 4 bytes).

Pointer Arithmetic

Operation	Result
`*p_arr`	Move `p_arr` by 0 bytes, dereference to get `arr[0]`
`*(p_arr + 1)`	Move `p_arr` by 4 bytes, dereference to get `arr[1]`
`*(p_arr + 2)`	Move `p_arr` by 8 bytes, dereference to get `arr[2]`

Pointer Arithmetic

Note that `arr` also decays into a pointer to the first element that shares the same memory address as `p_arr`, hence operations that can be done with `arr` can also be done with `p_arr`

```
#include <iostream>

int main() {
    int arr[3] = {10, 20, 30};
    std::cout << arr[1] << std::endl; // 20

    std::cout << *(arr + 1) << std::endl; // 20

    int* p_arr = arr;
    std::cout << *(p_arr + 1) << std::endl; // 20
}
```

😎 Cool Fun Fact

We've learnt that we can access `arr[0]` via `*(arr + 0)`. Since addition is commutative, `*(arr + 0) == *(0 + arr)`, hence `0[arr]` works too!

```
#include <iostream>
int main()
{
    int arr[3] = {10, 20, 30};
    std::cout << 1 [arr] << std::endl;
    std::cout << arr[1] << std::endl;
}
```

Strings

Fundamentally, a string is simply an array of characters ending with a special character `\0` (i.e null terminator)

The usual pointer arithmetic rules that we've learnt above apply too.

```
#include <iostream>

int main() {
    char str[] = "hello";
    std::cout << str[0] << std::endl;        // 'h'
    std::cout << *(str + 1) << std::endl;    // 'e'

    char* str2 = "world";
    std::cout << str2[0] << std::endl;        // 'w'
    std::cout << *(str2 + 1) << std::endl;    // 'o'
}
```

⚠️ Warning

Notice when we compile, we get a warning:

```
1:16:51 [~/Desktop/cpp]
❯ clang++ -std=c++17 main.cpp -o hello-world
main.cpp:160:18: warning: ISO C++11 does not allow conversion from string literal to 'char *' [-Wwritable-strings]
    char *str2 = "world";
                 ^
1 warning generated.

1:17:01 [~/Desktop/cpp]
❯ ./hello-world
h
e
w
o
```

Any ideas why?

Why the warning for `char* str2` but not `char str[]`

`"hello"` is a string literal stored in read-only memory.

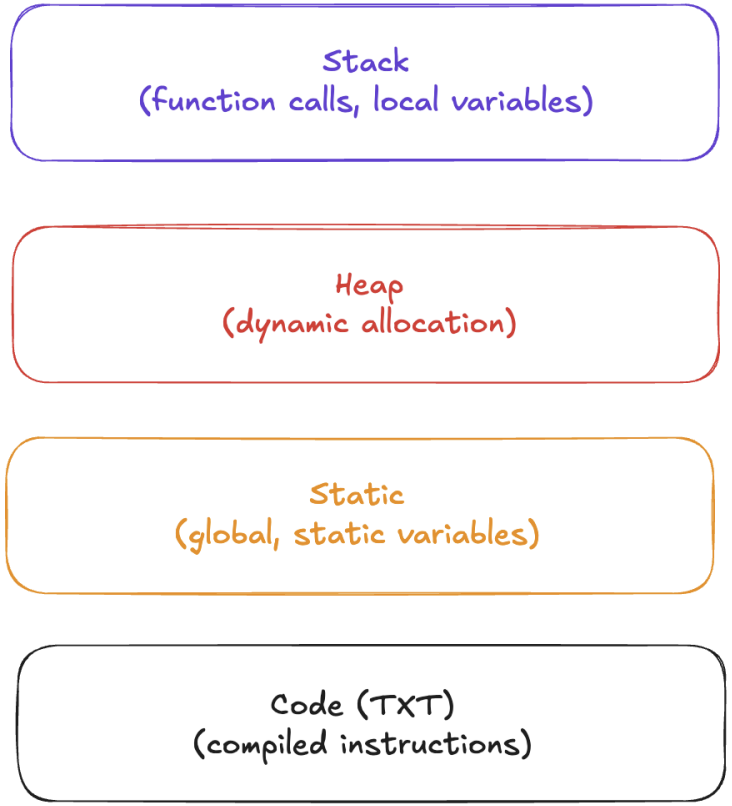
`char[]` has no issues since it copies the string literal onto its own array on the stack (more on this later).

`char *` is pointing to the same string literal in read-only memory. Writing to it causes undefined behaviour hence the warning.

Memory Layout

Virtual Memory

Variables can either be created or destroyed. They can also exists on different locations (i.e the **stack**, **heap** or **static** storage)



Stack vs Heap

Stack

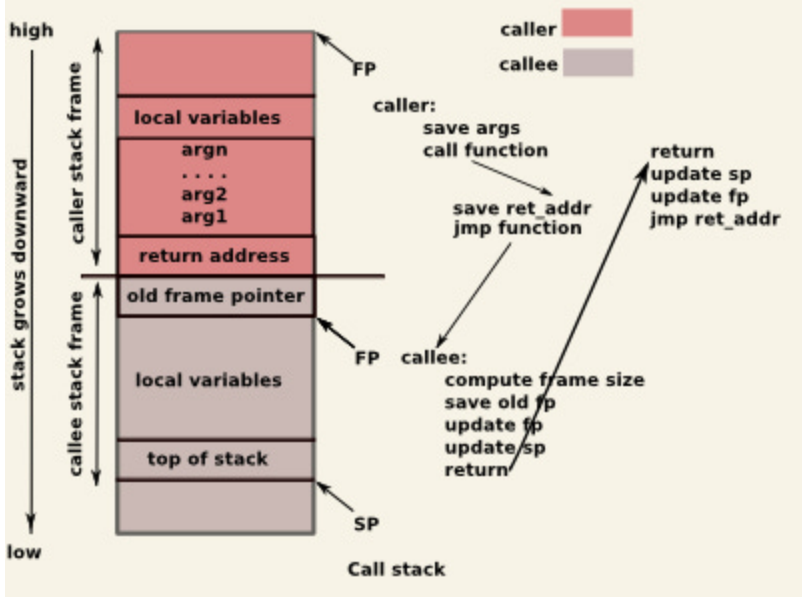
Variables created (i.e allocated) on the stack possess automatic storage duration. This means they are:

- allocated when execution enters their scope
- deallocated when out of scope

Why is stack allocation faster

Allocating (and deallocating) variables on the stack is fast as it mainly involves incrementing (and decrementing) the stack pointer.

This is also the reason why uninitialized variables on the stack possess garbage values because the memory they occupy is not automatically zeroed.



Stack Allocation

To jog your memory, all the previous variable declarations in **Variables** and **Pointers** were all allocated on the stack.

```
int main () {
    int x = 5;
    int* p = &x;
    int arr[3] = {10, 20, 30};
    // this copies the string literal onto the array on the stack
    char str[] = "hello";

    // the pointer str2 is on stack
    // the string literal world is stored in ROM
    const char* str2 = "world";
}
```

Stack Allocation

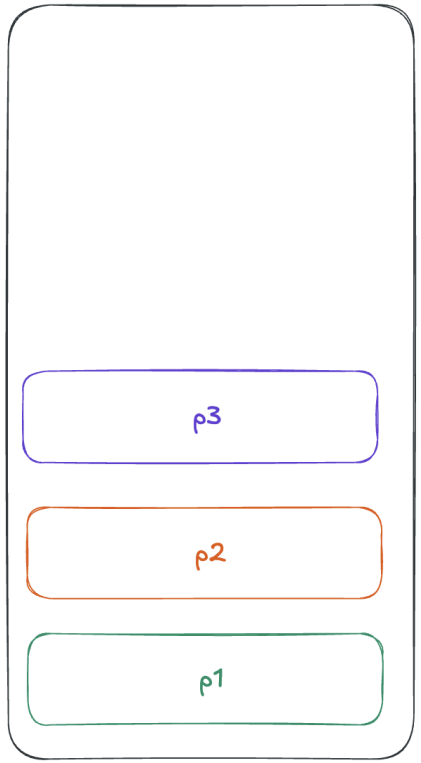
```
#include <iostream>

class Person {
...
...
};

int main()
{
    Person p1(10);
    Person p2(15);
    Person p3(20);
}
```

Code Output

```
22:44:09 [~/Desktop/cpp]
❯ clang++ main.cpp -o hello-world

22:44:12 [~/Desktop/cpp]
❯ ./hello-world
creating a person with 10
creating a person with 15
creating a person with 20
destroying a person with 20
destroying a person with 15
destroying a person with 10
```

Explanation

Observe that similar to the `Last-In-First-Out (LIFO)` behaviour of literal stack data structure, objects are constructed in order of declaration. They are also destroyed in order of reverse order.



Stack Allocation

Note that the behaviour is tied to scope, objects are destroyed as soon as it exits the scope.

```
int main()
{
    Person p1(10);
    {
        Person p2(15);
    }
    Person p3(20);
}
```

Heap

While the stack is fast, it does suffer from a few limitations:

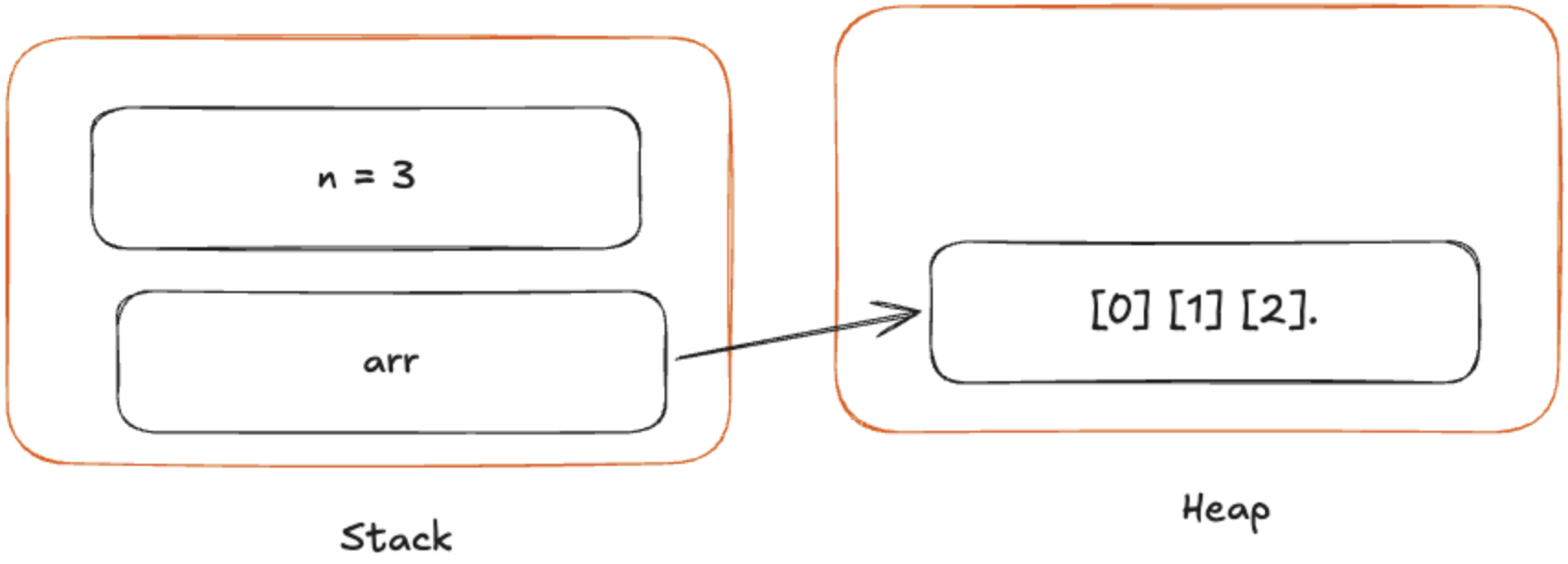
- Size of the objects must be known / determinable at compile time
- The lifetime of objects is tied to scope

Heap Allocation

You can use a `new` keyword to allocate memory on the heap. Unlike the stack, the heap memory is not tied to scope. It persists until it is explictly deallocated with `delete`.

```
#include <iostream>

int main() {
    int n;
    std::cin >> n;
    int* arr = new int[n];
    for (int i = 0; i < n; i++)
        arr[i] = i;
    for (int i = 0; i < n; ++i)
        std::cout << arr[i] << " ";
}
```

Visual Aid



⚠️ Fatal Flaw

There's a memory issue in the below code snippet. Can you spot it ?

```
#include <iostream>

int main() {
    int n;
    std::cin >> n;
    int* arr = new int[n];
    for (int i = 0; i < n; i++)
        arr[i] = i;
    for (int i = 0; i < n; ++i)
        std::cout << arr[i] << " ";
}
```

⚠️ Fatal Flaw

We've just encountered a common pitfall of allocating variables on the heap. The above code snippet is plagued by a memory leak.

The memory allocated using `new` is never deallocated using `delete`. This means the allocated memory remains reserved after after it's no longer needed.

We can fix the code by appending `delete[]` arr; to the back of the program like so:

```
#include <iostream>

int main() {
    ...

    // delete the array when you don't need it!
    delete[] arr;
}
```

⚠️ Dangling Pointers

Another common pitfall that programmers often stumble upon is the issue of dangling pointers. This refers to the situation when we try to access memory that already has been deallocated.

```
#include <iostream>
int main() {
    int *p = new int(5);
    std::cout << *p << std::endl;
    delete p;
    std::cout << *p << std::endl;
}
```

Running the Code

The above is an example of **undefined behavior**. For me, it prints 0 but the C++ standard makes no guarantees about what happens.

```
23:36:17 [~/Desktop/cpp]
❯ clang++ -std=c++17 -Wall -Wextra -pedantic-errors main.cpp

23:36:19 [~/Desktop/cpp]
❯ ./hello-world
5
0
```

It could also potentially print 5, print garbage values or crash through a segmentation fault.

Memory Painpoints

Ownership

Fundamentally, ownership answers the question: **Who is responsible for cleaning up this piece of memory?**

As we've seen earlier, memory can be allocated on two distinct regions (i.e the stack and the heap)

Stack

When variables are allocated on the stack, there's no burden on us to deallocate the memory. C++ handles it for us automatically!

```
int main() {
    int x = 10;
} // x is automatically destroyed by end of this scope
```

Memory Leak Exercise

```cpp
#include <iostream>
int* spawnScoreBonus() {
    return new int(100);
}

int* generatePlayerHealth() {
    int* health = new int(50);
    delete health;
    return health;
}

int main() {
    int* score = spawnScoreBonus();
    std::cout << "Player gained score: " << *score << std::endl;

    int* health = generatePlayerHealth();
    std::cout << "Player health: " << *health << std::endl;

    int* enemyHp = new int(200);
    int* archerTarget = enemyHp;
    int* knightTarget = enemyHp;
    delete knightTarget;
    delete archerTarget;

    return 0;
}
```

⚠️ Memory Issue 1

The `spawnScoreBonus()` function allocates a new block of memory containing `100` . Ownership of this object is transferred from `spawnScoreBonus()` to `main()`.

This means the onus is on `main()` to delete the object but `score` is never deallocated, leading to a **memory leak**!

⚠️ Memory Issue 2

In the `generatePlayerHealth()` function, `health` is deleted before being returned. This means the returned pointer is pointed to free'ed memory.

However, we are still trying to access the value `health` is pointing to in the `main()` function, leading a **dangling-pointer / use-after-free**.

⚠️ Memory Issue 3

Both `archerTarget` and `knightTarget` both point to the same integer of `200`. So both pointers behave like owners and both delete the same blokc of memory.

This leads to a **double-free** scenario where both owners think they own the shared resource.

The Beginning of An End

As we have seen earlier, even in a small piece of code, there can be many subtle memory issues. Imagine how many of such bugs exist in larger codebases!

To tackle this, modern C++ (since C++11) introduced something a little smarter...

Smarter Pointers

As we've seen, we humans are pretty dumb and inept at managing memory.

Here's where smart pointers come in! They are smarter abstractions introduced in C++11 to help us manage memory automatically.

Unique Pointers

Semantically, unique pointers convey the idea there is **exactly only one owner** of the resource being pointed to.

Unique pointers:

- cannot be copied
- can be moved
- are automatically deleted when it goes out of scope

Code Snippet

```
#include <iostream>
#include <memory>

int main() {
    std::unique_ptr<int> score = std::make_unique<int>(100);
    std::cout << *score << std::endl;
} // automatically deleted here
```

Transferring Onwnership

```
#include <memory>

void consume(std::unique_ptr<int> p) {
    // owns the pointer
}

int main() {
    auto x = std::make_unique<int>(50);
    consume(std::move(x));  // transfer ownership
}
```

Shared Pointers

Both `p1` and `p2` point to the same object. Memory for the integer `10` is only free'ed when both `p1` and `p2` goes out of scope.

```
#include <iostream>
#include <memory>

int main() {
    auto p1 = std::make_shared<int>(10);
    auto p2 = p1;  // shared ownership

    std::cout << *p1 << std::endl; // 10
    std::cout << *p2 << std::endl; // 10
}
```

Reference Counting

Internally, a `shared _ptr` uses a reference count to track when it shoud deallocate. We can see the live reference count by invoking `use_count()`.

```
#include <iostream>
#include <memory>

int main() {
    auto p1 = std::make_shared<int>(10);
    std::cout << p1.use_count() << std::endl; // 1

    auto p2 = p1;
    std::cout << p1.use_count() << std::endl; // 2
}
```

Tackling Our Original Code

Let's try relying on some of the above abstractions to write better, memory-safe code.

```
#include <iostream>
int* spawnScoreBonus() { return new int(100); }
int* generatePlayerHealth() {
    int* health = new int(50);
    delete health;
    return health;
}
int main() {
    int* score = spawnScoreBonus();
    int* health = generatePlayerHealth();
    std::cout << "Player health: " << *health << std::endl;
    int* enemyHp = new int(200);
    int* archerTarget = enemyHp;
    int* knightTarget = enemyHp;
    delete knightTarget;
    delete archerTarget;
}
```